
[CS3704] Intermediate Software Design and Engineering

Dr. Chris Brown
Virginia Tech
9/15/2026

Software Design and Architecture

Design Overview
UML Diagrams
High-Level Design
Low-Level Design

Learning Outcomes

By the end of the course, students should be able to:

- **Understand software engineering processes, methods, and tools used in the software development life cycle (SDLC)**
- Use techniques and processes to create and analyze requirements for an application
- **Use techniques and processes to design a software system**
- Identify processes, methods, and tools related to phases of the SDLC
- Explain the differences between software engineering processes
- Discuss research questions and current topics related to software engineering
- Create and communicate about the requirements and design of a software application

Announcements

- **PM1.1 due tomorrow (9/16) before midnight!**
- **PM1.2.1 due Thursday (9/24) *before class!!!***
 - Lightning talk demo slides
 - Project 1 demos in class on 9/24
- Submit preference for 10/6 guest lecture
- Ut Prosim opportunities
 - More added to #Ut Prosim Mattermost channel

Warm-Up: Review

- What software engineering process would you use to re-implement the Blacksburg Transit App. Why?
Pick one and explain...
- Which git command pushes code changes to your local repository?
git commit
- What is the difference between requirements analysis and requirements specification?

Analysis: understanding requirements (what the system must achieve)

Specification: describing requirements (how system meets reqs. *from user's perspective*)

TODO: Discuss with a partner and *submit responses on Canvas for attendance!*

Goal: decide the structure of the software and the hardware configurations that support it.

- The *how* of the project
- How individual classes and software components work together in the software system.
 - User interfaces, **structural elements, and data**
- *Software Artifacts:* UML (i.e. class diagrams, ER diagrams), Architecture Decision Records, etc.

Running Lecture Example

Standup Bot: A software bot to automatically schedule standup meetings between teammates. 

- *Scrum master*: Professional to ensure scrum processes for development team

TODO: Complete a stand-up meeting!

- What I did.
- What I need to do next.
- What is blocking me.

Diagrams

- Diagrams are essential in software development...

Understanding existing code

Design review

Explain to external members

Ad-hoc meetings (whiteboard)

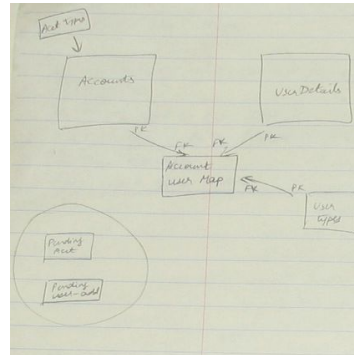
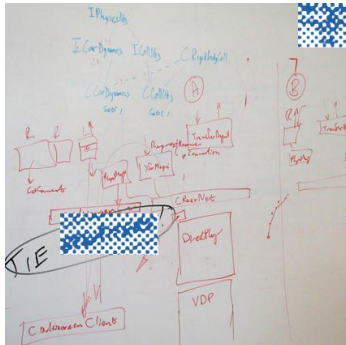
Onboarding

Hallway art

Refactoring plans

Explain to colleagues & newcomers

Documentation



Types of Diagrams

Software Requirements & Design

- *Class Diagrams**
- *ER Diagrams**
- Sequence Diagrams
- Use Case Diagrams
- ...

Interface Design (next class)

- Wireframes
- Flow Maps/Navigation Maps
- Storyboards
- ... *and many more!*

Specific diagram types [\[edit\]](#)

A	L
• Activity diagram used in UML 8/9 and SysML	• Ladder diagram
B	• Line of balance • Link grammar diagram
• Bachman diagram • Booch – used in software engineering • Block diagram • Block Definition Diagram (BDD) used in SysML • Business & IT Diagram (B&IT) – used in business and IT modelling	M • Martin ERD • Message Sequence Chart • Mind map – used for learning, brainstorming, memory, visual thinking and problem solving
C	N
• Carroll diagram • Cartogram • Category theory diagrams • Cause-and-effect diagram • Circuit diagram • Class diagram – from UML 1/9 • Collaboration diagram – from UML 2.0 • Communication diagram – from UML 2.0 • Commutative diagram • Component diagram – from UML 3/9 • Composite structure diagram – from UML 2.0 • Concept map • Constellation diagram • Context diagram • Control flow diagram • Contour diagram • Cordier diagram • Cross functional flowchart	• N2 • Nassi–Shneiderman diagram or structogram – a representation for structured programming • Nomogram • [Network connection]
D	O
• Data model diagram • Data flow diagram • Data structure diagram • Dendrogram • Dependency diagram • Deployment diagram – from UML 9/9 • Dot and cross diagram • Double bubble map – used in education • Drakon-chart	• Object diagram – from UML 2/9
E	P
• Entity-Relationship diagram (ERD) • Event-driven process chain • Euler diagram • Eye diagram – a diagram of a received telecommunications signal • Express-G • Extended Functional Flow Block Diagram (EFFBD)	• Package diagram from UML 4/9 and SysML • Parametric diagram from SysML • PERT • Petri net – shows the structure of a distributed system as a directed bipartite graph with annotations • Phylogenetic tree - represents a phylogeny (evolutionary relationships among groups of organisms) • Piping and instrumentation diagram (P&ID) • Phase diagram used to present solid/liquid/gas information • Plant Diagram • Pressure volume diagram used to analyse engines • Pourbaix diagram • Process flow diagram or PFD – used in chemical engineering • Program structure diagram
F	R
• Family tree • Feynman diagram • Flow chart • Flow process chart • Fusion diagram	• Radar chart • Radial Diagram • Requirement Diagram Used in SysML • Rich Picture • R-diagram
	S
	• Sankey diagram – represents material, energy or cost flows with quantity proportional arrows in a process network. • Sentence diagram – represents the grammatical structure of a natural language sentence. • Sequence diagram from UML 8/9 and SysML • SDL/GR diagram – Specification and Description Language. SDL is a formal language used in computer science. • Smith chart • Spider chart

UML

- Unified Modeling Language (UML) is the standard for modeling object-oriented software.
 - Currently on version 2.0
 - Typically depicted with two types of diagrams
 - Structural (i.e. class diagrams)
 - Behavioral (i.e. use case and sequence diagrams)

How is UML Really Used?

“UML has been described by some as ‘the lingua franca of software engineering’. Evidence from industry does not necessarily support such endorsements. How exactly is UML being used in industry...” [Petre]

<u>NONE!</u>	70%
SELECTIVE	22%
AUTOMATIC CODE GEN	6%
RETROFIT	2%
WHOLE	0%

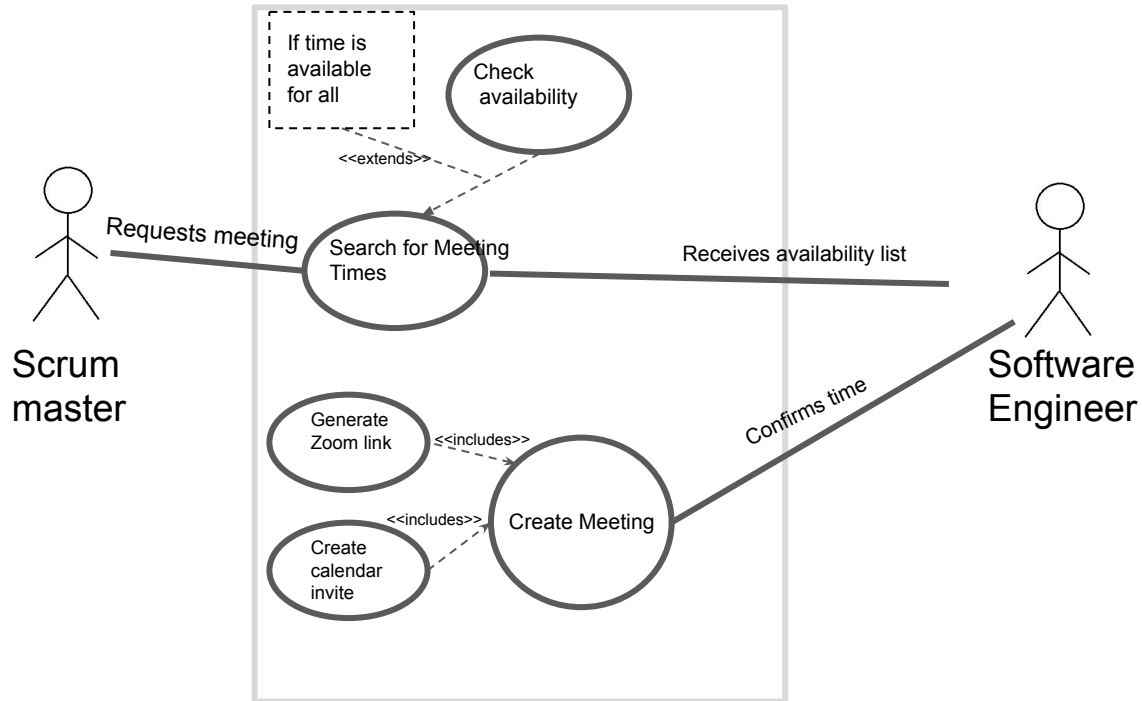
Of those that reported using it...

TABLE II. ELEMENTS OF UML USED BY THE 11 ‘SELECTIVE’ USERS.

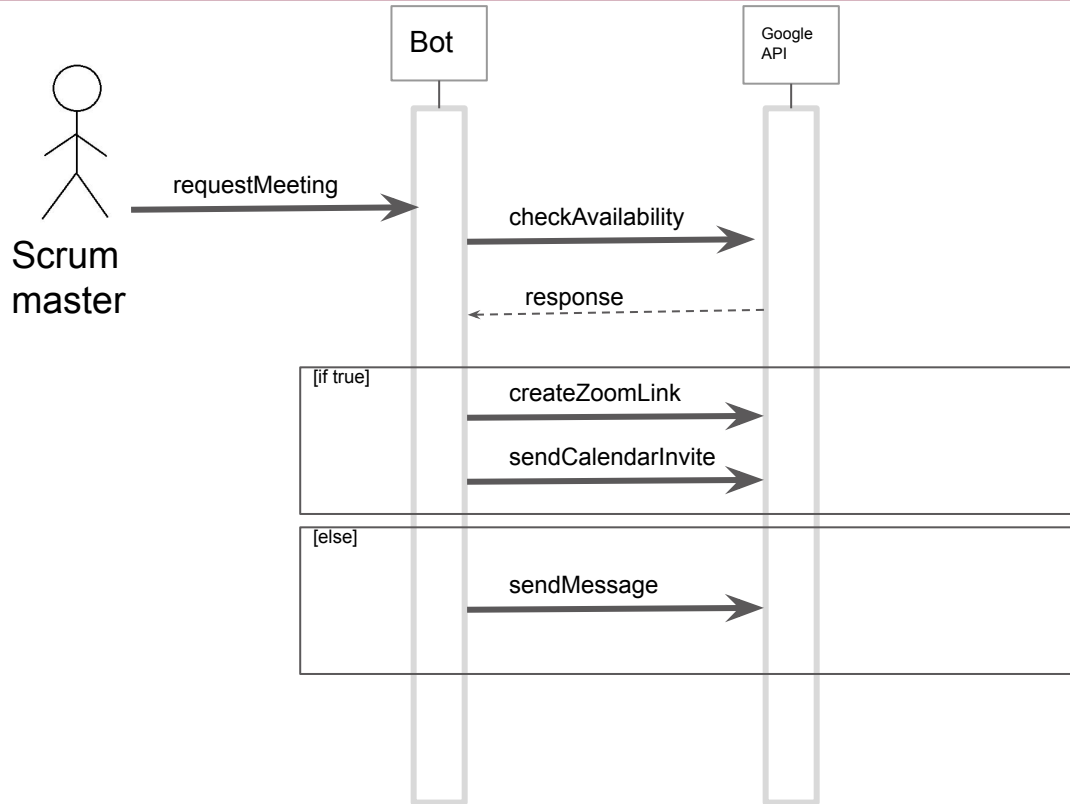
UML diagrams	Number of users	Reported to be used for...
Class diagrams	7	structure, conceptual models, concept analysis of domain, architecture, interfaces
Sequence diagrams	6	requirements elicitation, eliciting behaviors, instantiation history
Activity diagrams	6	modeling concurrency, eliciting useful behaviors, ordering processes
State machine diagrams	3	
Use case diagrams	1	represent requirements

***TODO: How do you design coding projects?
What is your experience with UML?***

Review: Use Case Diagram

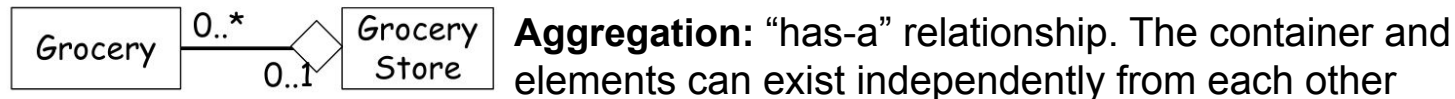
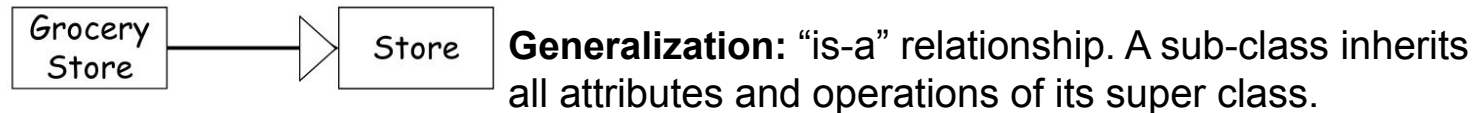
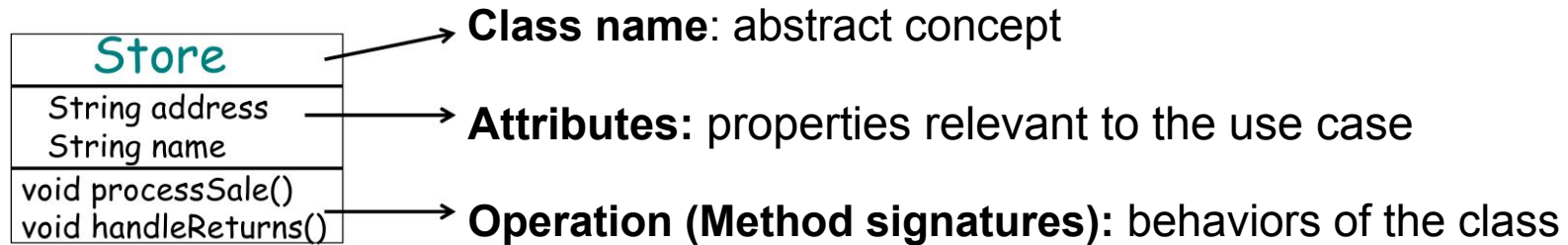


Review: Sequence Diagram

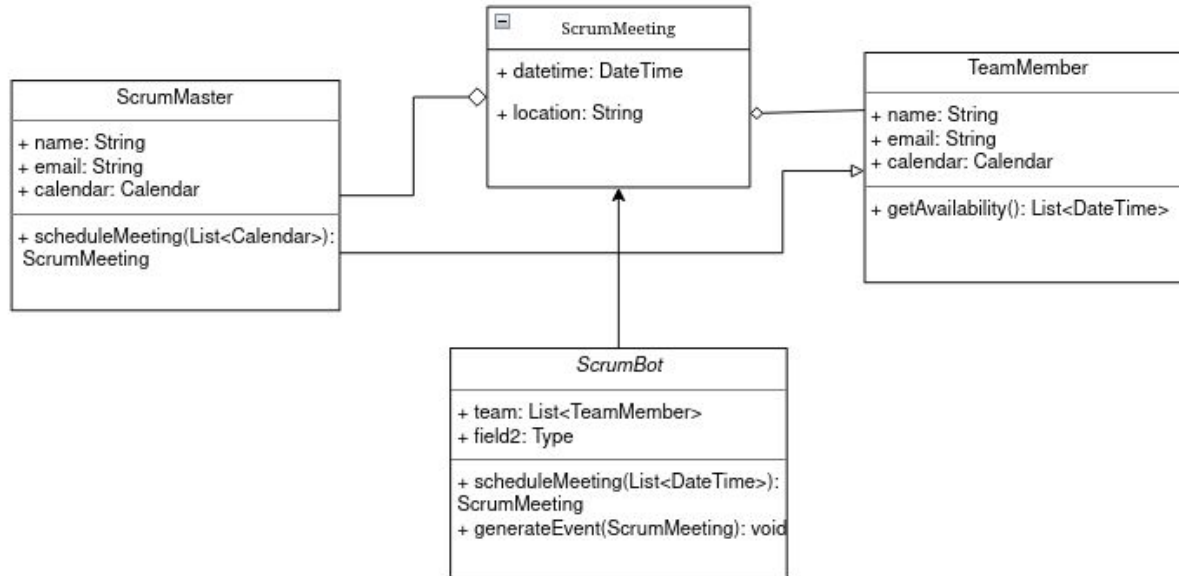


Class Diagram

A visual representation of main objects and their relations for a system.



Class Diagram Example



Database Design

Modern software is collecting and processing increasing amounts of data.

- What is a database?
 - A system that stores data, and lets you create, read, update, and delete the data
- Why use databases?
 - Databases plays a critical role in applications
 - Corrupted data => execution failure
 - Poor data organization => poor performance
 - A poorly designed database allows developers and users to put in arbitrary data *or access data without authorization!*

Relational Databases

A digital database with a collection of tables.

- Each table contains rows and columns, with a unique key for each row
- Each entity type described in a database has its own table
- Each row represents an instance of the entity
- Each column represents an attribute

Table: USERS

userId (PK)	name	email	timezone
101	Alice Dev	alice@dev.com	UTC-5 (EST)
102	Bob ScrumMaster	bob@scrum.com	UTC-5 (EST)
103	Charlie Dev	charlie@dev.com	UTC-8 (PST)

Table: TEAMS

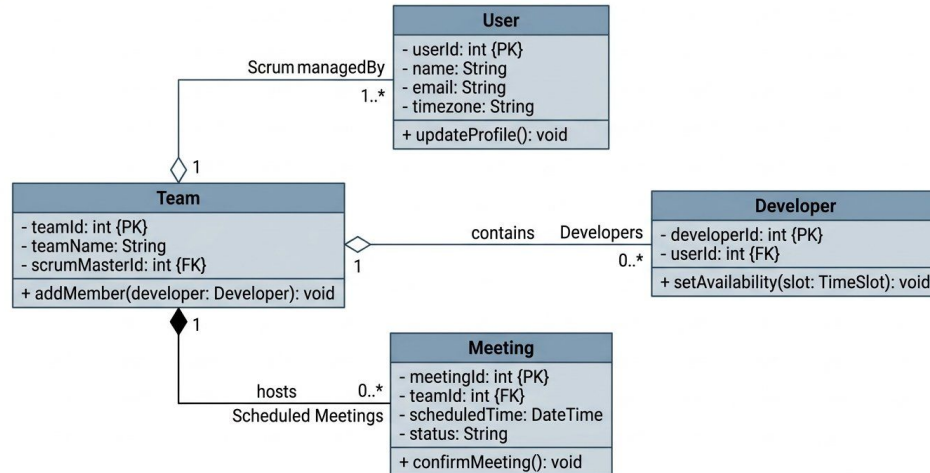
teamId (PK)	teamName	scrumMasterId (FK -> Users)
50	Frontend Alpha	102

Primary Key/Unique Key: to uniquely specify a tuple in a table

Foreign Key: an attribute in a relational table that matches the primary key column of another table. It can be used to cross-reference tables.

Entity-Relationship Diagram

- Entity-relationship (ER) diagrams are similar to semantic object modelings (i.e., class diagrams)
 - Focus is more on data and less on code structure



Software Design

- **High-level: Architecture design**
 - Define major components and their relationship
- **Low-level: Detailed design**
 - Decide classes, interfaces, and implementation algorithms for each component

Architectural (High-Level) Design

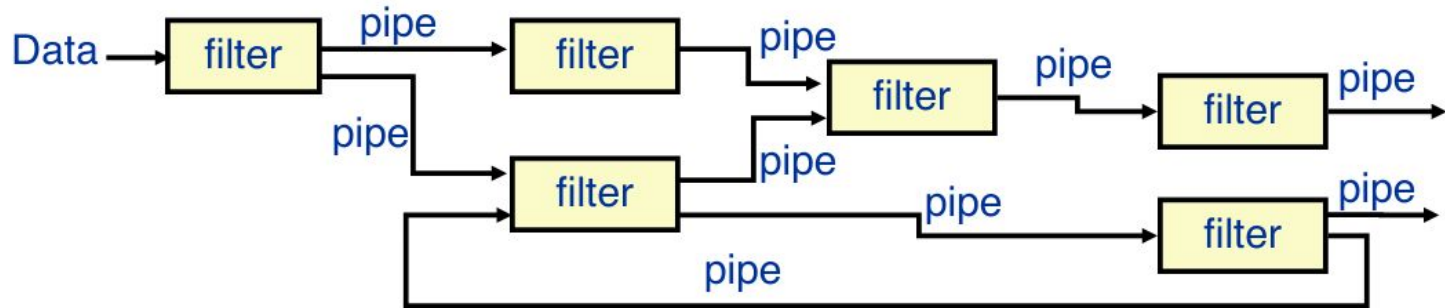
- Design overall shape & structure of system
 - the components
 - their externally visible properties
 - their relationships
- **Goal:** choose architecture to reduce risks in software construction and meet requirements.

Common Architecture Patterns

- 1) Pipe & Filter Architecture
- 2) Event-based Architecture
- 3) Layered Architecture
- 4) Microservices

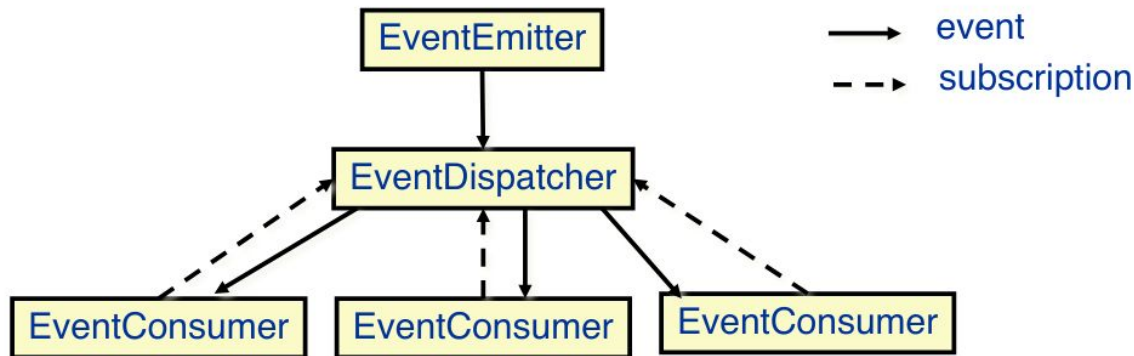
Pipe and Filter

- A pipeline contains a chain of data processing elements
 - The output of each element is the input of the next element



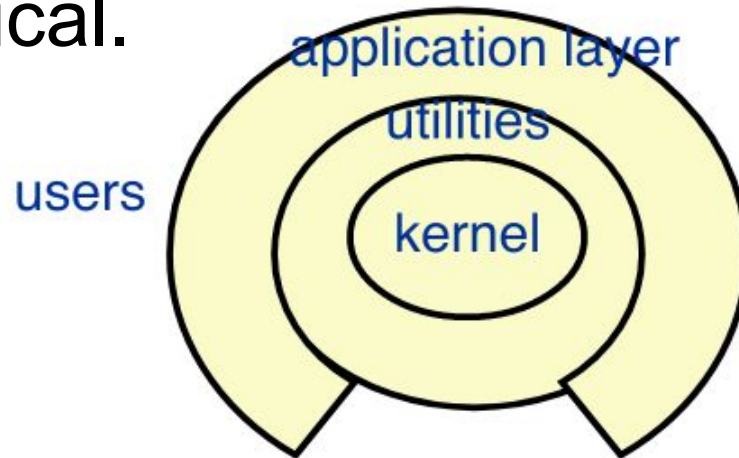
Event-Based Architecture

- Promotes the production, detection, consumption of, and reaction to events
- *Event-driven programming*

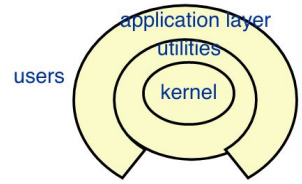


Layered/Tiered Architecture

- Multiple layers are defined to allocate responsibilities of a software product
- The communication between layers is hierarchical.

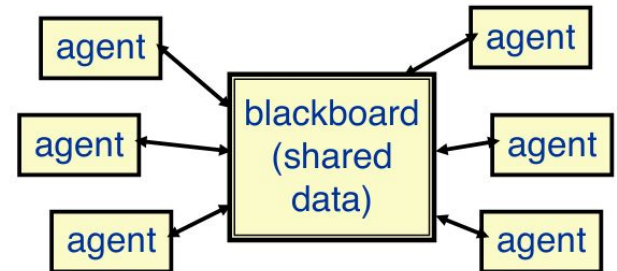
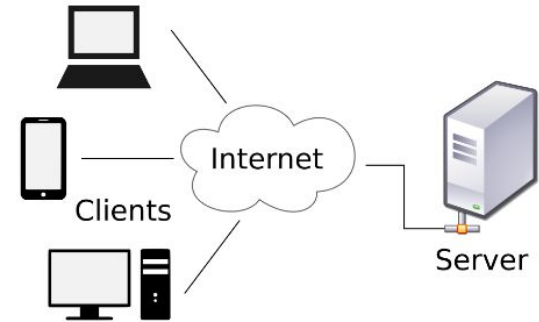


Layered/Tiered (cont.)



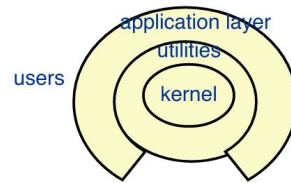
Variants: 2-layer architecture

- Client-Server Architecture
 - Multiple hardwares
 - Partition tasks or workloads between providers and consumer
- Data-centric Architecture
 - Data accessed frequently
 - Notifications to subscribers



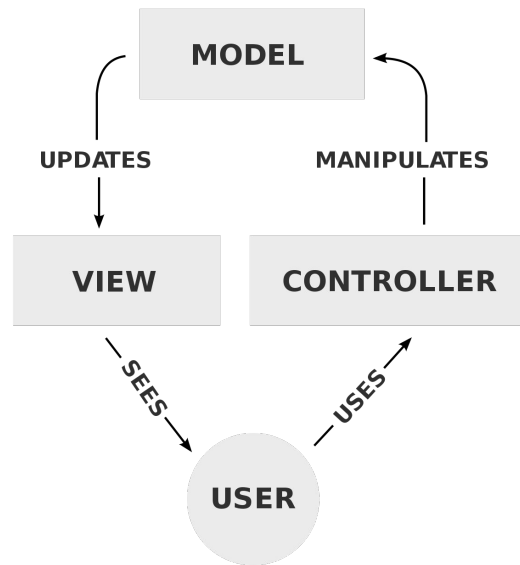
Ex) most web-based applications, version control system, etc.

Layered/Tiered (cont.)



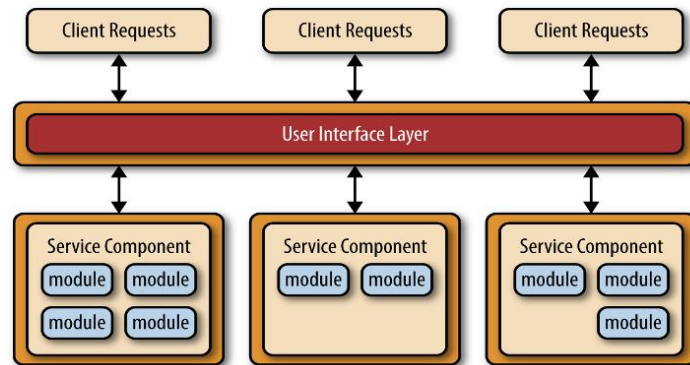
Variants: 3-layer architecture

- Model-View-Controller
 - UI to interact with users
 - Store and retrieve information as needed



Microservices

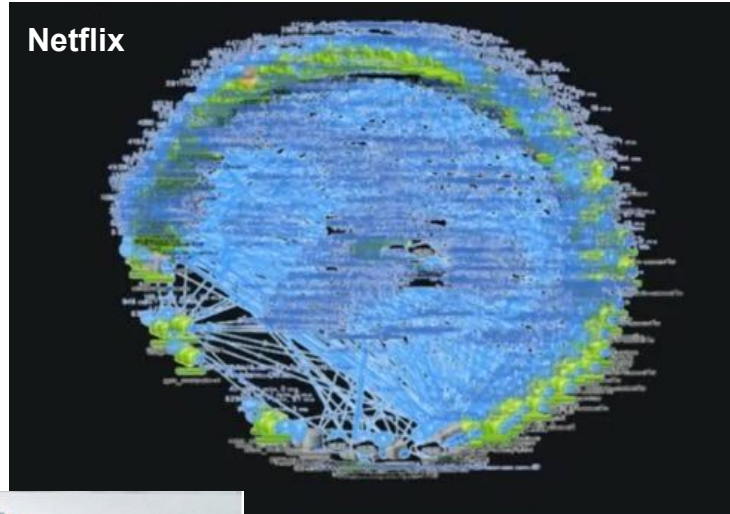
The microservices pattern involves creating multiple applications—or *microservices*—that work interdependently. Although each microservice can be developed and deployed independently, its functionality is interwoven with other microservices.



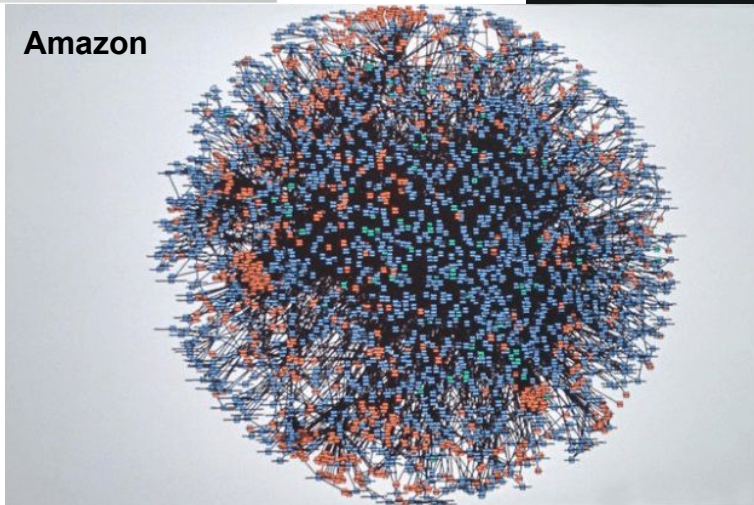
Uber



Netflix



Amazon



Low-Level Design

Goal: To decompose subsystems into modules

Two main approaches:

1. Procedural:

- System is decomposed into functional modules which accept input data and transform it to output data

2. Object-oriented

- System is decomposed into a set of communicating objects (achieves procedural + data abstractions)

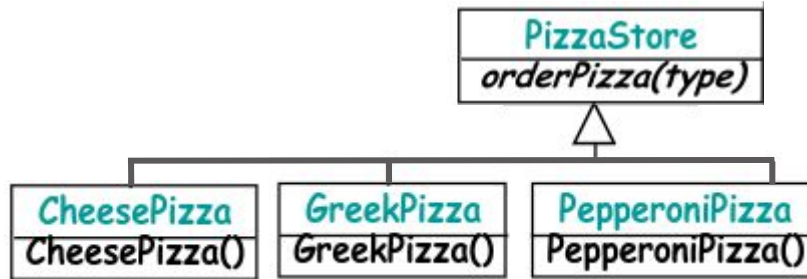
S.O.L.I.D. Principles of OOD

- **S** – Single-responsibility principle
- **O** – Open-closed principle
- **L** – Liskov substitution principle
- **I** – Interface segregation principle
- **D** – Dependency Inversion Principle

More details in 10/15 guest lecture!

Example

Creating software for a Pizza Store 🍕

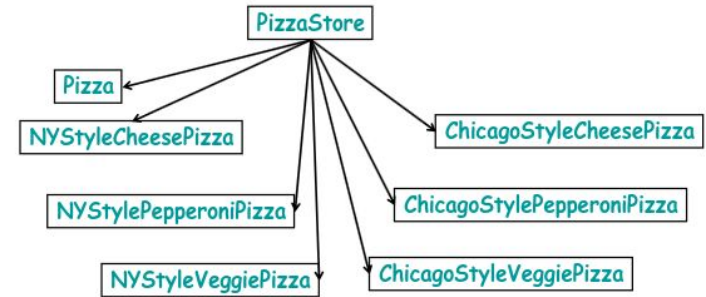


Discuss: What is wrong with this design?

Example

Problems...

- Client (PizzaStore) invokes different pizza constructors among other responsibilities
- New pizza types may be added
 - Clam, Veggie, etc.
- New order types may be added
 - In Store, Delivery, etc.
- Original pizza types may be removed
 - Greek, etc.
- Different styles of pizza
 - Chicago, NY, Stuffed crust, etc.
- ...



Design Patterns (i.e. Low-Level Design)

Design patterns are descriptions of *communicating objects and classes* that are customized to solve a **general** design problem in a particular context.

The design pattern identifies the participating **classes and instances**, their **roles and collaborations**, and the distribution of **responsibilities**.

Design Patterns (cont.)

Why design patterns?

- Apply working solutions to approaches
- Based on the implementations of many systems
- Capture and pass on the knowledge of experienced designers
 - Useful for inexperienced
 - Communicating about design

Gang of Four

Design Patterns

Elements of Reusable
Object-Oriented Software

Erich Gamma
Richard Helm
Ralph Johnson
John Vlissides



Foreword by Grady Booch



Design Pattern Families

Creational

Concerned with the process of object creation

- Increases flexibility and reuse of code

Structural

Deal with the composition of classes or objects

- Organizing different classes and modules to form larger structures or add new functionality

Behavioral

Characterize the ways in which classes or objects interact and distribute responsibility

- Algorithms and assignment of responsibilities between objects

Creation Patterns

- **Abstract Factory:** Creates an instance of several families of classes
- **Builder:** Separates object construction from its representation
- **Factory Method:** Creates an instance of several derived classes
- **Object Pool:** Avoid expensive acquisition and release of resources by recycling objects that are no longer in use
- **Prototype:** A fully initialized instance to be copied or cloned
- **Singleton:** A class of which only a single instance can exist

Structural Patterns

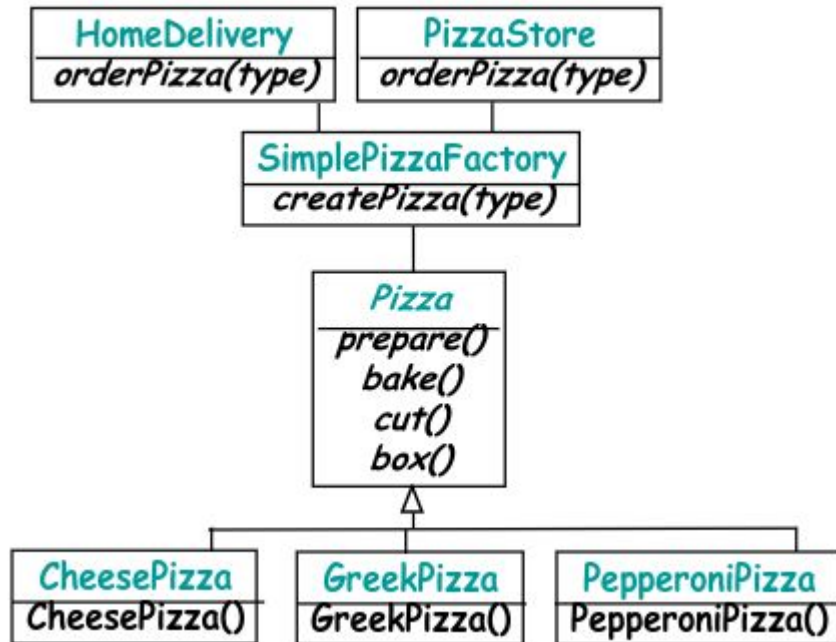
- **Adapter:** Match interfaces of different classes
- **Bridge:** Separates an object's interface from its implementation
- **Composite:** A tree structure of simple and composite objects
- **Decorator:** Add responsibilities to objects dynamically
- **Facade:** A single class that represents an entire subsystem
- **Flyweight:** A fine-grained instance used for efficient sharing
- **Private Class Data:** Restricts accessor/mutator access
- **Proxy:** An object representing another object

Behavioral Patterns

- **Chain of responsibility:** A way of passing a request between a chain of objects
- **Command:** Encapsulate a command request as an object
- **Interpreter:** A way to include language elements in a program
- **Iterator:** Sequentially access the elements of a collection
- **Mediator:** Defines simplified communication between classes
- **Memento:** Capture and restore an object's internal state
- **Null Object:** Designed to act as a default value of an object
- **Observer:** A way of notifying change to a number of classes
- **State:** Alter an object's behavior when its state changes
- **Strategy:** Encapsulates an algorithm inside a class
- **Template method:** Defer the exact steps of an algorithm to a subclass
- **Visitor:** Defines a new operation to a class without change

Example Solution: Pizza Factory

- Encapsulate object creation

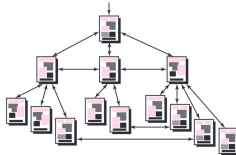
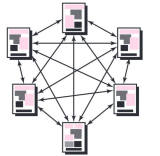
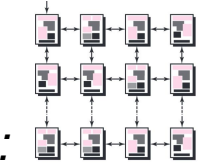


Discuss: What is wrong with this design?

- Increased maintenance effort
- Pizza class violates Single Responsibility Principle
- The cut and box methods do not depend on Pizza type
- ...

Design Disclaimer

- The design of your program will differ based on project goals, team, and domain.
- Examples:
 - Web app
 - *“There are essentially two basic approaches to [web] design: the artistic ideal of expressing yourself and the engineering ideal of solving a problem for a customer” [Nielsen]*
 - Aesthetics, layout, graphic design, content, navigation,...
 - Mobile app
 - Still concerned with aesthetics...along with device type (smartphone, tablet, etc.), more complex interactions, different power and storage capabilities, etc.



Putting it All Together



Plan-Driven

- Diagrams and Modeling (i.e., Class Diagrams)
- Software Architecture Document (SAD) 🥵

Iterative

- Design-while-coding 🥵
- Architecture Decision Records (ADRs)

Architecture Decision Records (ADRs)

A short document capturing critical software design choices.

- Captures *why* behind design decisions, not just *how*
- Typically kept alongside source code
- Needs to be reviewed and approved by the team
- Helps provide understanding for why code is built a certain way

Architecture Decision Record (ADR) Template

Title	Short name of the decision to be made
Date	Date when the decision was made
Context	Explanation of the problem
Decision	Clear statement of the decision made
Alternatives	Description of alternative solutions that were considered
Consequences	Implications of the decision, positive or negative
Participants	Names of team members how made the decision
Status	The current status of the decision, such as "proposed," "accepted," "rejected," or "superseded."

LLMs in Software Design

Software Architecture

- LLMs are primarily used for four main tasks:
 - Reference architectures
 - Classification and Detection
 - Extraction and Generation
 - Assistants

“We found that most of them [articles on LLMs for software architecture] propose automated approaches that use LLMs end-to-end, suggesting that **LLMs are capable of addressing complete architectural tasks.**” [Schmid]

LLMs in Software Design (cont.)

Low-Level Design

- LLMs are less reliable for low-level design tasks. [Pan]

TABLE II

OVERALL COMPARISON OF DIFFERENT LLMs ON LINE COMPLETION AND FUNCTION GENERATION WITH AND WITHOUT PRIOR KNOWLEDGE OF DESIGN PATTERN. EACH RESULT BOX SHOWS LINE COMPLETION ON THE LEFT AND FUNCTION GENERATION ON THE RIGHT, SEPARATED BY A '/'.

Models	OpenAI Models				Qwen Models				Llama Models				DeepSeek_V2		Claude35		Other Models		Yi_34B		GLM_3_6B	
	GPT-4		GPT-4o		CodeQwen		Qwen_25_72B		Llama_31_405B		Llama_31_70B		Yes	No	Yes	No	Yes	No	Yes	No	Yes	No
Design Pattern Provided (Y/N)	Yes	No	Yes	No	Yes	No	Yes	No	Yes	No	Yes	No	Yes	No	Yes	No	Yes	No	Yes	No	Yes	No
Code Similarity (%)	24.68/23.23	22.99/31.05	37.35/32.93	51.31/53.71	5.4/11.20	5.50/10.79	20.27/39.67	17.84/42.75	37.02/28.08	41.02/44.70	26.59/29.67	26.49/41.55	18.01/19.89	12.11/30.90	45.75/32.45	48.16/33.63	29.83/10.02	40.16/39.58	8.17/14.65	9.30/18.20	7.99/10.65	8.61/10.21
Edit Similarity (%)	23.91/26.08	21.21/32.08	33.30/32.77	43.72/54.27	6.2/12.21	5.54/13.74	20/39.11	17.72/42.34	32.58/25.75	34.78/45.73	24.36/28.97	24.98/43.21	18.48/21.01	13.54/31.41	37.35/29.91	41.50/33.40	28.31/16.51	35.22/38.62	8.90/19.17	9.96/23.99	11.39/15.60	12.60/16.08

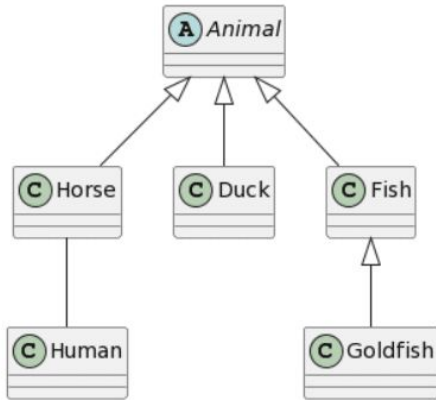
TABLE III

COMPARISON OF LLMs ON FUNCTION GENERATION FOR VARIOUS DESIGN PATTERNS WITH AND WITHOUT PRIOR KNOWLEDGE OF THE CODE FILE'S DESIGN PATTERN. EACH RESULT BOX SHOWS CODE SIMILARITY (%) ON THE LEFT AND EDIT SIMILARITY (%) ON THE RIGHT, SEPARATED BY A '/'.

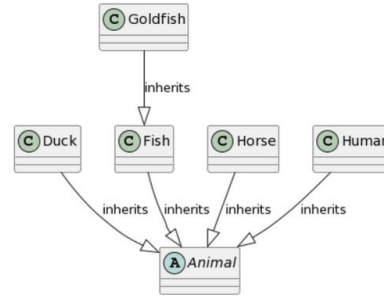
Models	OpenAI Models				Qwen Models				Llama Models				DeepSeek_V2		Claude35		Other Models		Yi_34B		GLM_3_6B	
	GPT-4		GPT-4o		CodeQwen		Qwen_25_72B		Llama_31_405B		Llama_31_70B		Yes	No	Yes	No	Mistral_123B	Yes	No	Yes	No	
Design Pattern Provided (Y/N)	Yes	No	Yes	No	Yes	No	Yes	No	Yes	No	Yes	No	Yes	No	Yes	No	Yes	No	Yes	No	Yes	No
Singleton	33.64/39.89	13.20/32.10	36.15/32.77	42.73/50.79	2.74/5.47	2.48/4.98	52.57/44.11	35.19/36.03	35.08/33.22	32.52/58.02	37.37/34.83	39.60/49.90	18.55/14.65	19.77/19.78	22.82/21.85	12.91/33.17	6.63/11.40	62.12/55.73	23.50/21.19	13.78/22.92	19.07/22.12	7.94/15.22
Factory	34.31/34.23	38.73/30.64	20.22/23.34	47.92/52.73	14.57/15.52	8.99/11.65	25.32/31.21	31.50/29.83	32.55/32.66	38.14/54.32	29.24/28.42	36.80/34.45	14.76/20.60	23.06/25.77	32.47/33.00	15.53/19.52	7.72/17.70	25.30/26.07	6.55/15.98	8.92/12.92	6.55/15.16	2.69/8.58
Builder	6.68/21.07	13.41/26.04	28.12/41.23	33.89/43.64	14.41/16.52	15.13/20.81	10.87/21.00	9.31/17.46	15.90/24.81	34.07/40.58	6.39/19.64	13.94/22.96	3.22/11.00	9.47/15.80	11.70/24.41	17.47/27.47	8.57/17.39	9.76/25.77	10.91/30.16	5.13/15.32	4.95/13.52	
Decorator	34.48/35.37	41.15/31.48	33.10/32.20	62.03/62.57	17.75/24.17	13.24/23.33	41.55/36.47	61.06/61.04	32.67/24.83	52.00/49.47	32.01/32.19	53.69/52.46	30.04/36.05	52.81/52.34	36.84/33.21	49.56/53.39	13.81/20.76	37.55/35.70	19.45/26.41	25.59/33.44	24.10/23.82	30.20/27.96
Proxy	35.48/34.06	27.18/20.18	36.90/37.00	36.22/32.08	27.69/28.86	9.77/14.50	36.48/35.62	36.70/32.98	29.37/27.22	35.16/30.65	28.38/24.52	35.08/30.62	21.08/15.53	33.44/31.18	26.94/23.56	26.23/19.31	5.72/14.46	34.03/29.30	4.71/15.56	6.01/4.95	12.99/15.49	2.82/12.13
Strategy	29.60/28.45	43.32/43.18	37.28/35.75	65.08/60.22	7.39/8.44	13.76/11.65	37.91/37.32	48.61/47.48	25.03/23.83	44.20/44.36	20.86/21.84	44.03/46.95	16.98/20.03	34.95/34.12	35.59/31.12	41.57/38.08	10.28/17.36	54.15/50.15	19.65/19.64	19.22/21.19	10.26/15.66	18.51/23.00
Template	33.62/33.33	54.19/46.37	62.20/51.98	65.59/60.11	10.49/7.06	27.37/21.76	66.17/61.46	62.40/57.79	49.88/37.91	57.13/50.01	44.44/36.74	66.45/66.09	30.41/26.21	36.75/35.58	59.31/43.98	48.59/38.56	16.69/18.04	63.12/50.14	2.51/6.36	3.09/7.10	8.50/14.86	1.95/11.44
Observer	7.67/5.86	5.08/5.30	15.46/10.29	76.48/70.35	7.96/6.58	6.73/4.92	48.23/44.82	77.50/64.55	13.79/10.30	70.38/62.31	28.48/20.82	43.07/40.17	16.23/11.56	43.73/29.23	45.53/29.67	36.48/23.06	12.21/10.99	48.02/45.74	42.08/36.55	41.13/34.52	14.19/9.68	9.02/7.12
Command	16.30/16.03	31.19/36.99	24.80/19.02	51.71/52.11	7.55/6.00	1.65/9.98	64.51/55.23	39.66/46.10	24.07/14.96	35.94/45.65	49.09/37.33	36.70/42.28	24.88/18.05	37.82/45.08	27.24/18.61	49.60/40.90	11.08/13.43	37.39/44.05	12.67/10.79	32.66/38.14	3.68/12.40	8.59/15.18
Bridge	6.78/17.21	6.67/15.98	37.07/40.71	37.58/40.12	11.59/11.01	12.05/12.66	37.14/39.17	30.15/28.63	38.94/27.63	20.12/20.08	31.08/36.29	21.13/18.23	32.71/30.17	25.15/24.73	28.14/28.91	26.54/22.94	3.02/16.23	32.19/32.84	8.46/21.26	10.85/18.09	5.15/7.99	4.03/7.24
Facade	11.29/11.67	39.81/41.41	32.54/31.91	36.92/56.33	5.22/5.12	3.71/5.62	29.16/31.38	39.27/39.57	30.78/25.05	53.04/50.49	37.99/38.90	59.49/61.47	15.44/21.76	19.06/23.43	36.57/34.69	41.95/37.29	7.76/13.57	29.14/32.45	15.13/14.72	18.29/24.81	8.01/12.65	3.98/15.25
Composite	11.99/18.30	24.57/25.52	35.32/32.79	63.52/64.07	4.68/3.68	9.23/16.08	58.40/56.79	58.96/53.79	19.76/23.66	39.68/40.56	32.29/25.48	50.59/51.61	24.81/18.08	32.78/27.15	34.08/32.76	27.06/28.30	16.85/20.56	51.22/52.05	17.55/16.55	35.39/41.27	9.64/14.34	4.59/17.10

LLMs in Software Design (cont.)

LLMs are terrible at generating UML diagrams



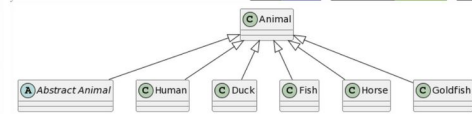
ChatGPT4: 2 mistakes



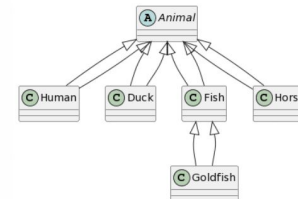
CogVLM: 10 mistakes



Microsoft Copilot: 3 mistakes



Bard: 3 mistakes



Next Class...

No Class Activity
UI Design

Announcements

- **PM1.1 due tomorrow before midnight!**
- **PM1.2.1 due Thursday (9/24) before class!!!**

Warm-Up: Review

- What software engineering process would you use to re-implement the Blacksburg Transit App. Why?
Pick one and explain...
- Which git command pushes code changes to your local repository?
git commit
- What is the difference between requirements analysis and requirements specification?

Analysis: understanding requirements (what the system must achieve)

Specification: describing requirements (how system meets reqs. *from user's perspective*)

TODO: Discuss with a partner and *submit responses on Canvas for attendance!*

References

- RS Pressman. *“Software engineering: a practitioner's approach”*.
- Anand Butani. *“5 essential patterns of software architecture”*. Red Hat, 2020.
<<https://www.redhat.com/en/blog/5-essential-patterns-software-architecture#client-server>>
- Cast Software *“What is Software Architecture?”*.
<<https://www.castsoftware.com/glossary/what-is-software-architecture-tools-design-definition-explanation-best>>
- K.D. Cooper, L. Torczon, *“Engineering a Compiler”*. Theo Mandel. *“Golden Rules of User Interface Design”*.
<<https://theomandel.com/resources/golden-rules-of-user-interface-design/>>
- <<https://medium.com/swlh/ordering-food-and-the-mvc-architecture-d5cbf3859d60>>
- Simon Foucher. “Introduction to Software Engineering: Unit 9 – Architectural Design Approaches”. McGill, ECSE-321.
- <https://refactoring.guru/design-patterns>
- Ian Sommerville. “Software Engineering”.
- Emil Vassev, Joey Paquet. “Java Design Patterns”. 2006-2012. Concordia University
- Ashish raap Singh. “S.O.L.I.D. Principles Explained with Code.” AlgoMaster Blog, 2024.
<https://blog.algomaster.io/p/solid-principles-explained-with-code>
- Guy Kawasaki. “The Only 10 Slides You Need in a Pitch”. 2020
<https://guykawasaki.com/the-only-10-slides-you-need-in-your-pitch/>
- Noah Parsons. “The 11 Slides You Need to Have in Your Pitch Deck for 2025”. LivePlan, 2025.
<https://www.liveplan.com/blog/funding/slides-you-need>
- Yoonsuck Choe. “SOLID Principles for Object-Oriented Design”. CSCE 315, Texas A&M. 2020.
- Robert Martin. “Principles of OOD”. <http://butunclebob.com/ArticleS.UncleBob.PrinciplesOfOod>

References (cont.)

- Aaron Conrardy. “From Image to UML: First Results of Image Based Class Diagram Generation Using LLMs”. Modeling Languages. <https://modeling-languages.com/image-to-uml-with-llm/>
 - Larissa Schmid, Tobias Hey, Martin Armbruster, Sophie Corallo, Dominik Fuchß, Jan Keim, Haoyu Liu, Anne Koziolk. “Software Architecture Meets LLMs: A Systematic Literature Review”. Arxiv:2505.16697, 2025.
 - Zhenyu Pan, Xuefeng Song, Yunkun Wang, Rongyu Cao, Binhua Li, Yongbin Li, Han Liu. “Do Code LLMs Understand Design Patterns”. Arxiv:2501.04835, 2025.
 - Modern Analyst. “What is an Architecture Decision Record (ADR)?”
<https://www.modernanalyst.com/Careers/InterviewQuestions/tabid/128/ID/6594/What-is-an-Architecture-Decision-Record-ADR>
 - Jasmin Jahic, Ashkan Sami. “State of Practice: LLMs in Software Engineering and Software Architecture”. International Conference on Software Architecture Companion (ICSA-C), 2024.
 - Na Meng and Barbara Ryder
 - Chris Parnin
 - Sarah Heckman
-